

Lekcja 06 - dataclass i Pydantic w pipeline Python

2026-08-14

Data Engineering Course

■ dataclass i Pydantic

W Python Data Engineering pipeline

```
raw dict -> Pydantic validation -> dataclass Order  
-> metrics -> output
```

Cel: zrozumieć różnice między zwykłą klasą, `dataclass` i Pydantic.

■ Problem

Dane z pliku albo API przychodza jako raw payload.

Najczesciej widzisz cos takiego:

```
record = {  
    "order_id": "1001",  
    "status": "Completed",  
    "total_amount": "120.50",  
    "source": "api",  
}
```

To jest `dict`.

Pytanie: czy mozemy mu ufac?

■ dict

dict przechowuje dane jako klucz -> wartosc.

```
record["status"]  
record["total_amount"]
```

Plusy:

```
prosty  
naturalny format po CSV/JSON  
łatwy do wypisania
```

Minus:

```
nie pokazuje kontraktu rekordu
```

■ Problem z dict

Ten kod działa tylko wtedy, gdy dane są poprawne:

```
amount = float(record["total_amount"])
```

Ale payload może być taki:

```
{"order_id": "1002", "total_amount": "not-a-number"
}
```

Albo taki:

```
{"status": "completed", "total_amount": "120.50"}
```

Brakuje `order_id`.

■ Zwykła klasa

Mozesz napisac zwykla klase:

```
class Order:
    def __init__(self, order_id: str, status: str,
total_amount: float) -> None:
        self.order_id = order_id
        self.status = status
        self.total_amount = total_amount
```

To dziala.

Ale dla prostych rekordow jest duzo powtarzalnego kodu.

■ Co daje zwykła klasa

Zwykła klasa jest dobra, gdy obiekt ma zachowanie.

Przykład:

```
class Order:
    def __init__(self, order_id: str, status: str,
total_amount: float) -> None:
        self.order_id = order_id
        self.status = status
        self.total_amount = total_amount

    def is_completed(self) -> bool:
        return self.status == "completed"
```

Klasa = dane + zachowanie.

■ Minus zwykłej klasy

Dla prostych rekordów pisziesz dużo boilerplate:

```
__init__  
self.field = field  
__repr__  
__eq__  
czasem immutable handling
```

Boilerplate to kod, który musisz napisać, ale nie niesie dużo logiki biznesowej.

■ dataclass

dataclass generuje nudną część za Ciebie.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Order:
    order_id: str
    status: str
    total_amount: float
    source: str
```

To jest dobry format dla poprawnego rekordu w pipeline.

■ class vs dataclass

```
class
    piszesz __init__ sam
    dobra, gdy obiekt ma więcej zachowania
    więcej kodu dla prostych rekordów

dataclass
    Python generuje __init__, repr, eq
    dobra do struktur danych
    mniej kodu, bardziej czytelny kontrakt
```

W pipeline często chcesz `dataclass` dla rekordu po walidacji.

■ dataclass z metoda

`dataclass` może mieć metody.

```
@dataclass(frozen=True)
class Order:
    order_id: str
    status: str
    total_amount: float
    source: str

    @property
    def is_completed(self) -> bool:
        return self.status == "completed"
```

To nadal jest proste i czytelne.

■ frozen=True

```
@dataclass(frozen=True)
class Order:
    order_id: str
    status: str
```

frozen=True znaczy: po utworzeniu nie zmieniaj obiektu.

Dlaczego to pomaga?

```
rekord po walidacji jest stabilny
trudniej przypadkiem zmienic status w srodku
pipeline
latwiej debugowac transformacje
```

■ Ale dataclass nie wystarczy

`dataclass` nie jest mocna walidacja raw inputu.

```
Order(  
    order_id="1001",  
    status="unknown",  
    total_amount=-10.0,  
    source="api",  
)
```

To technicznie może się utworzyć.

Ale biznesowo rekord jest podejrzany.

Pydantic waliduje raw payload w runtime.

```
from pydantic import BaseModel, Field

class OrderPayload(BaseModel):
    order_id: str
    status: str
    total_amount: float = Field(ge=0)
    source: str = "unknown"
```

Pydantic sprawdza dane wtedy, gdy przychodzi z zewnatrz.

■ Co sprawdza Pydantic

Dla `OrderPayload`:

```
order_id musi istniec  
status musi byc tekstem  
total_amount musi dac sie zamienic na float  
total_amount musi byc >= 0  
source ma default "unknown"
```

To jest kontrakt wejścia do pipeline.

Mozesz dopisać reguły biznesowe.

```
ALLOWED_STATUSES = {"completed", "cancelled", "pending", "refunded"}

@field_validator("status")
@classmethod
def normalize_and_validate_status(cls, value: str)
-> str:
    normalized = value.strip().lower()
    if normalized not in ALLOWED_STATUSES:
        raise ValueError("unknown status")
    return normalized
```

Tu status jest normalizowany i sprawdzany.

■ Pydantic vs dataclass

Pydantic

granica pipeline

sprawdza raw dict z CSV/JSON/API

mowi: accepted albo rejected

dataclass

srodek pipeline

reprezentuje poprawny rekord

wygodny do transformacji i metryk

To sa rozne narzedzia.

Nie musisz wybierac jednego.

```
CSV/JSON
|
v
raw dict
|
v
OrderPayload.model_validate(record)
|
v
Order dataclass
|
v
calculate_completed_revenue(orders)
|
v
pipeline_output.json
```

To jest realny wzorzec pipeline.

■ Accepted i rejected

Pipeline powinien rozdzielić rekordy:

```
accepted: list[Order] = []  
rejected: list[RejectedRecord] = []
```

Rejected record powinien mieć powód:

```
@dataclass(frozen=True)  
class RejectedRecord:  
    record: dict[str, object]  
    reason: str
```

Bez reason nie wiesz, co naprawić.

■ Przykład accepted

Raw payload:

```
{"order_id": "1001", "status": " Completed ", "total_amount": "120.50"}
```

Po Pydantic:

```
OrderPayload(order_id="1001", status="completed",  
total_amount=120.5, source="unknown")
```

Po konwersji:

```
Order(order_id="1001", status="completed",  
total_amount=120.5, source="unknown")
```

■ Przykład rejected

Raw payload:

```
{"order_id": "1006", "status": "completed", "total_amount": "-12.00"}
```

Pydantic odrzuca:

```
total_amount: Input should be greater than or equal to 0
```

Do outputu zapisujesz:

```
record + reason
```

```
def calculate_completed_revenue(records:
list[Order]) -> float:
    revenue = 0.0
    for record in records:
        if record.is_completed:
            revenue = revenue +
record.total_amount
    return revenue
```

Ta funkcja nie martwi się już o raw JSON.

Dostaje tylko poprawne `Order`.

■ Dlaczego to jest ważne w DE

Data Engineering to nie tylko policzenie sumy.

Musisz wiedzieć:

```
czy dane przyszły w dobrym kształcie,  
ile rekordów zaakceptowano,  
ile odrzucono,  
dlaczego odrzucono,  
czy metryki licza się tylko na accepted records.
```

To buduje zaufanie do pipeline.

Testy powinny sprawdzać:

```
valid payload przechodzi  
missing order_id jest rejected  
negative amount jest rejected  
unknown status jest rejected  
status robi sie lowercase  
accepted payload staje sie Order dataclass  
revenue liczy tylko completed orders  
output JSON ma accepted_count i rejected_count
```

To są testy kontraktu, nie tylko składni.

Pytanie:

How would you use dataclasses and Pydantic in a Python data pipeline?

Odpowiedz:

I use Pydantic at the boundary of the pipeline to validate raw payloads from files or APIs. If the payload is valid, I convert it into a dataclass that represents the domain record. Transformations and metrics operate on dataclasses, not raw dictionaries. Invalid payloads are written to rejected records with error reasons. This keeps validation, domain modeling and business logic separate.

raw dict
 dane z zewnatrz, nie ufamy

Pydantic model
 waliduje input

dataclass
 reprezentuje poprawny rekord w kodzie

transform functions
 licza metryki na accepted records

rejected records
 pokazuja, co i dlaczego odpadło

■ Closing check

Powiedz na głos:

Zwykła klasa daje dane i zachowanie, ale wymaga więcej kodu.
dataclass jest lepszy do prostych rekordów po walidacji.
Pydantic waliduje raw dane z pliku, API albo JSON.
W pipeline używam Pydantic na wejściu, dataclass w środku.
Rejected records muszą mieć reason.